

supportgitlab-com or supportself-managed and supportmanager Slack channel about the work to be done with a runbook link

- [] Verify that any rollback plan can be execute by an engineer from every region
- [] Has a dry-run been performed before the call?
- [] Do we have a way to create logs from the actions performed.

Runbook -Tasks

When possible, link the task name with a project, handbook page, which documents it.

Task	Outcome	Duration

Example

Task	Outcome	Duration
Confirm the users provided state	Success	10m
Modify user with MUser console method	Success	1h15m

Rollback plan

In the rollback plan, we provide a link for the steps to revert the actions performed. If we have a situation where the rollback requires only a simple explanation, it can be added to the runbook directly.

Post-call checklist

- [] Inform on supportgitlab-com or supportself-managed and supportmanager slack channel the about the work completion, the current state, and if any follow up is necessary with documentation links.

Complete RunBook Example

Runbook name	Acme - Enforce 2FA on all groups
Runbook description	Acme Inc. requires us to enforce 2FA on all of their 2500 groups
DRI	@johndoe
Schedule at	2020-10-10 - 23:30 PST
Meeting URL	example.zoom.com/my/example.meeting
GitLab Issue	https://gitlab.com/gitlab-com/demo/example/example-project/-/issues/1

GitLab Contacts

Name	Role	Description
------	------	-------------

John Doe	Technical Execution	Technical execution during the change
Jane Smith	Communication & Technical assistance	In charge of updates and to support the technical execution role

Customer Contacts

Name	Role	Description
Tom Blogs	Systems Engineer	Work verification and support
harry Page	IT Manager	Coordination from customer side

Pre-call checklist

plain

[x] - Inform on supportgitlab-com or supportself-managed and supportmanager slack channel about the work to be done with a runbook link

[x] - Verify that any rollback plan can be execute by an engineer from every region

[] - Has a dry-run been performed before the call?

Dry-run cannot be completed due to change requirements

[x] - Do we have a way to create logs from the actions performed.

Runbook - Tasks

Task	Outcome	Duration
Confirm the users provided state		
Remove 2FA from user list		
Review users with errors		
Update issue with outcome		

Rollback plan

The method used for this change takes an optional parameter status which is disable by default, to rollback we need to execute the same method adding a parameter `status: 'enable'.

Post-call checklist

[x] - Inform on supportgitlab-com or supportself-managed and supportmanager slack channel the about the work completion, the current state, and if any follow up is necessary with documentation links.

SECTION: support/workflows/support-sales-escalations.md

Purpose

There will be instances when Support will need to escalate a ticket to Sales/Customer Success, or bring it to their attention. This page lists some of these common scenarios and workflows.

> Note: If you are from Sales/CS and looking to escalate a ticket to Support Management, please use the I want to escalate a ticket workflow instead.

Escalation Scenarios Mapping

Role: Account Owner (AO)

An Account Owner is usually the customer's main point of contact for any renewal and expansion discussions.

Use the finding the customer's account owner workflow to locate the account owner. If there is no one listed, check to see if you can find the latest opportunity owner instead. If neither are available, reach out in the relevant channel without tagging anybody.

|Escalation Scenario|Channel|Relevancy|

|--|--|

|Add a customer to an organization |Slack: account-management|Needs Org Workflow|

|Pass a lead to Sales|Zendesk: cc @AM, Slack: sales for new business, account-management for renewals|Ticket Workflow: License and Renewals queue|

| Professional Services engagement request|Slack: account-management @mention AO | Ticket Workflow: All queues|

Role: Customer Success Manager (CSM)

A Customer Success Manager (formerly known as a Technical Account Manager or TAM) typically manages the customer's relationship with GitLab. A comprehensive list of services provided by them is listed here.

Since CSMs are assigned based on certain criteria, you might not find a CSM listed for every customer. For those that do have a CSM assigned, follow the same process as above on SFDC, but look at the Customer Success Manager field instead.

|Escalation Scenario|Channel|Relevancy|

|--|--|

|Make CSM aware of Emergency and/or High Priority tickets|Zendesk: cc @CSM, Slack: @mention CSM|Ticket Workflow: All queues|

|Involve CSM if a customer requested an escalation on the ticket|Zendesk: cc @CSM, Slack: @mention CSM|Ticket Workflow: All queues|

|Involve CSM if a customer files a ticket requesting upgrade assistance but there is no corresponding internal issue|Zendesk: cc @CSM, Slack: @mention CSM|Ticket Workflow: Self Managed|

|Involve CSM if we identify a need for training a user from customer's end, or, if customer requests for it directly|Zendesk: cc @CSM, Slack: @mention CSM|Ticket Workflow: All queues|

|Make CSM aware of tickets which require managing customer relationship due to expectations mismatch|Zendesk: cc @CSM Slack: @mention CSM|Ticket Workflow: All queues|

|Add a customer to an organization |Slack: account-management|Needs Org Workflow|

Scale Customer Success Engineer (CSE) engagement

Some organizations do not have a dedicated CSM/CSE but could benefit from Customer Success engagement. In the Zendesk notes, you'll see Customer Success Manager: CSM Scale. In cases where it is appropriate, you can reach out to the Account Owner to discuss opening a CSE Engagement Request.

Some examples on when this might be appropriate:

- Architectural best practices
- Discussing how other organizations like theirs use GitLab
- Migrations or the differences between GitLab.com and Self-Managed instances
- Addressing growth or blockers to feature adoption

Resources

- CSM Responsibilities and Services
- Customer Success Escalations Process

SECTION: support/workflows/support_hold.md

About account holds

An account hold happens if a customer doesn't pay their invoice on time. After several attempts to contact and reconcile the account, the accounts receivable team will:

- Chatter the AE responsible for the account
- Place the account on "Support Hold"

The result is that all tickets a customer on-hold attempts to create will get a reject with a message to contact their sales rep.

Once a hold is removed in Salesforce, the change will sync down to Zendesk and the account's contacts can open tickets.

Support Workflow for accounts on hold

Occasionally you may see an internal request for help with an account hold.

This is not a Support issue, and needs to be worked out with the accounts receivable team.

1. Let the internal requester know that this is an accounts issue due to an unpaid invoice.
1. Direct them to Chatter @[Shannon Williams] (or the current Accounts Receivable Manager) on the SFDC account
1. Let them know that once the situation is resolved with AR, the changes will sync to Zendesk.

Restoring access for accounts that have been removed from hold

Changes will sync from Salesforce to Zendesk relatively quickly after the change is made.

In the account view in Zendesk, you can look for the "Support Hold" checkbox - if it is unchecked, there is no hold on the account.

If you believe there is an issue with the sync, and Salesforce and Zendesk are showing different data you can Slack supportoperations.

SECTION: support/workflows/support_manager-on-call.md

Introduction

The Support Manager On-call helps coordinate responses to urgent and important situations that arise within the scope of delivering a quality experience to GitLab customers.

The Support Manager On-call is one of the rotations that make up GitLab Support On-call.

Expectations for Support Manager On-call

As part of GitLab Support on-call, Support Managers serve in a rotation. The support manager on-call is responsible generally for:

1. Ensuring customer emergencies are handled promptly and accurately, according to SLAs
1. Handling Support Ticket Attention Requests during Global Support Hours
1. Acting as a notification point for security incidents
1. Finding a Support Manager DRI for an Account Escalation
1. Help avoid SLA breaches. See Working on Tickets for details.

Note: You (or the CMOC/CEOC) may sometimes be required to contact GitLab users on behalf of another GitLab team (such as the SIRT team). Please follow the Sending Notices workflow to action these requests.

Handling customer emergencies and incidents

The Support Engineer on-call is the first responder for customer emergencies. Managers support this work as follows:

- Act as the next-tier escalation point for emergency pages that get missed (you will be notified automatically by PagerDuty).
- Respond to new emergency requests by helping the on-call engineer determine if the situation qualifies as an emergency or should be granted an exception
- Assist the on-call engineer in difficult communications with the customer, such as telling them that their request does not qualify as an emergency
- Be aware of ongoing emergencies and assist or lead in our initial response as appropriate
- During an emergency: find additional staff for subject-matter expertise; replace the on-call

engineer if needed; lead any Zoom call(s) as needed; hand off the emergency to the next on-call manager.

- Find additional staff when there are multiple emergencies.
- Convert customer emergencies into Account Escalations as needed.
- The CMOC should coordinate a Support Response when an incident requires a non-standard workflow or communication from Support. Support the CMOC by ensuring there is clarity and communication with relevant decision-makers in Support or other teams. You can also create the Support Response issue if this has not yet been done.

Situations that Might or Might not Be Emergencies

At times, an emergency page may come in for a situation that is not quite yet an emergency, but may quickly become one. In this situation, we want to assist the customer in preventing the situation from becoming an emergency. If this situation arises during working hours, the Support Engineer on-call may reach out for assistance. The on-call manager should respond by finding additional staff to handle the request as a high priority ticket that requires an immediate response. If this situation arises on a weekend, the Support Engineer on-call will reach out to the manager on-call if they are handling another emergency. In that case, the manager on-call should assist or attempt to find additional staff to assist.

See more examples of situations that might be emergencies and situations that are not emergencies.

Paging backup engineers on weekends in APAC

The APAC region has a pool of backup engineers that are available to reach out to during the weekend on-call hours, in the event that a concurrent emergency occurs.

If you are the Support Manager on-call and a concurrent emergency occurs, you will be paged by the Support Engineer On-call escalated via Pagerduty. You will then be responsible for checking the current situation and determining if the backup engineers need to be paged. If so, the Support Manager will then manually page the backup engineers. At this point, the backup engineers are all pinged. Only one backup engineer needs to acknowledge the page and lend assistance, and there is no expectation that backup engineers will be available to respond to a page.

To page the backup pool, you can:

1. Use the /pd trigger command in any Slack channel to create a new incident to notify the current list of support engineers; or
1. Create a + New Incident directly in PagerDuty.

When prompted, update:

- Impacted Service: Customer Emergencies - APAC Backup Pool
- Title: Duplicate emergency - ZD123456
- Description: Provide a brief summary of the emergency.
- Leave Assign To: and Priority: blank.

For further details, please refer to STM4583.

Handling Support Ticket Attention Requests during business hours

STARs (Support Ticket Attention Requests) are handled by the Support Manager on-call.

Your responsibilities are as follows:

1. Triage and investigate customer tickets and Internal Requests that are announced in the supportticket-attention-requests Slack channel.
1. Establish ownership and assignment for starred tickets.

You can use Support Team Skills by Subject to find appropriate engineers to assign.

A very high percentage of starred tickets involve licenses and renewals. For guidance in handling these, please see the Workflow for handling Plan/License Ticket Attention Requests.

NOTE: GitLab team members may attempt to draw attention to tickets in regular support Slack channels (supportself-managed, supportgitlab-com, supportleadership). Redirect the team member by responding to their post with only the :escalate: emoji, which will send an automated and anonymous reply describing the correct process.

NOTE: There are two other distinct situations, not discussed on this page:

1. Account Escalations / Escalated Customers
1. Emergencies that become Account Escalations

Mechanics of handling starred tickets

Some steps of STAR treatment are handled by bots and auto-responders. The text BOT is used below to show these steps.

1. When someone initiates a STAR using the STAR Form:
 1. BOT: Creates a new Issue in the STAR issue tracker.
 1. BOT: Slack announcement is posted to supportticket-attention-requests, with an @mention of the current on-call Support Manager's name.
 1. In many cases BOT adds an internal note to the Zendesk ticket and STAR Issue.
 1. Add the :eyes: emoji to the Slack thread to acknowledge you are looking at the STAR.
 1. In the Zendesk ticket, use the Support::Managers::Escalated Ticket macro to crosslink the STAR Issue and discussion thread, and tag the ticket.
 1. Assess the ticket, and the business case justifying the request (triage).
 - Questions to the initiator can be put in Slack (synchronous) or in the STAR Issue (asynchronous).
 - Final dispensation should be documented in the STAR Issue, because Slack history disappears.
 1. If you need input or help from engineers, initiate a new thread in supportgitlab-com, supportself-managed or supportlicensing-subscription.
 - @ mention the assigned engineer and any engineer who previously replied in the ticket, if they are working this day
 - Then return to the thread in supportticket-attention-requests and comment that all technical discussion is happening in the ticket (or in the new thread). This helps ensure all technical discussion stays in one channel/thread.

- When seeking an engineer to act as the DRI or otherwise move the ticket forward, it is best to identify a Support Engineer who is not on-call or already working on a starred ticket. This helps to ensure that the engineer who assists with the new starred ticket has enough bandwidth to prioritize it.

1. Resolve the STAR thread.

Unstarring a ticket - rejecting the request for additional attention

There are times when a STAR does not meet the threshold for additional attention. See the main STAR page for details. In such situations, return to the thread in supportticket-attention-requests and notify the initiator.

Resolving a STAR

A STAR is considered resolved when the correct next-step is identified and underway; it does not require the Zendesk ticket to be Solved or Closed.

When a STAR is resolved:

1. Apply a :green-check-mark: emoji to the notification in supportticket-attention-requests.

1. Update the associated STAR Issue with an appropriate comment, and Close it.

1. Consider applying an appropriate label, such as one of the scoped label examples below, to help categorize and track trends.

- ~Escalation::License-Issue : Identifies the core issue at hand resolves around licensing / subscriptions

- ~Escalation::Response-Time : Useful when the purpose of the request is to expedite a response to an issue or case

Finding a Support Manager DRI for an Account Escalation

In case an Account Escalation opens where support involvement is needed, it is the responsibility of the support manager on-call to find a Lead Support Manager. In case this is an ASE account (you can find this in the org notes) please ping Mike Dunninger (AMER), Ilia Kosenko (EMEA) or Wei Meng Lee (APAC).

Handling mid ticket feedback requesting manager contact during business hours

Mid-ticket feedback link -- each Public Comment from a GitLab Support Engineer or Manager has a link to a form where a customer can provide feedback or request contact from a manager while the ticket is open (introduced in issue 2913).

This feedback form creates issues in the customer feedback project, with a subject format of Positive/Negative/Neutral feedback for ticket nnnnnn, and is automatically assigned to the SSAT reviewing manager.

If the feedback is negative, there is an option to request manager contact (within 48hrs Mon-Fri). If this option is chosen, a Slack notification is sent to the supportticket-attention-requests channel.

The following action should be taken promptly:

1. Review feedback and ticket information and consider how urgent a reply is required. The reply should occur within 48hrs Mon-Fri. Replying sooner could prevent further customer

frustration and a STAR or emergency situation.

1. Reply via the ticket or email, as appropriate, sharing next steps and offer your Calendly link to schedule a video call.

1. Remain as DRI until the ticket is back on track.

1. Update the Feedback Issue as follows:

1. Add the text of any communication as a comment in the Feedback Issue.

1. Apply the label ~ssat-manager-contacted-customer.

1. /close the Feedback Issue; followup continues via previously chosen communication method.

1. After closing the Issue, if there are any additional actions that arise from your interaction with the customer, go back and note them in the Feedback Issue.

Act as a notification point for security incidents

When GitLab experiences a security incident, the Support Manager on-call is responsible for triaging and responding to customer communications stemming from the security incident. This may include involving the CMOC.

Upgrade assistance requests are currently triaged by engineers as part of the Working on Tickets but in some cases the triaging agent(s) may need assistance from Support management.

Example situations and potential solutions

- User requesting Upgrade Assistance outside of GitLab Support Hours
 - Reach out to your reports to determine whether any individuals would be open to shifting their working hours to accommodate the day/time being requested
 - Work with the end user to reschedule for another date/time that is more amenable to support staffing
- Last minute change in assignee availability
 - Work with available team members to determine if the upgrade assistance can be transferred to a new engineer and communicate the change(s) to the end user for awareness

Reassigning or swapping your on-call shift

If you will be unable to handle on-call for a few hours on a weekday due to being engaged in a customer call or otherwise, arrange for another manager to handle on-call responsibilities temporarily:

1. Ask a specific manager to cover; if that doesn't work,
1. Post a request in supportleadership for any manager to volunteer to cover.

To swap your on-call duty with someone, follow the steps listed under Swapping on-call duty.

Manually triggering a PagerDuty notification

At times, you may receive an escalation where the customer is reporting a situation that qualifies for emergency support under our definitions of support impact. In such cases you may elect to trigger an emergency directly, rather than asking the customer to open a new emergency ticket.

You can trigger a PagerDuty notification by using the Support::Managers::Trigger manual

emergency macro in Zendesk.

Alternatively, you can manually trigger a PagerDuty notification through PagerDuty itself.

Login to gitlab.pagerduty.com and select + New Incident from the upper right corner. Then fill out the form as follows:

- Impacted Service: Customer Support
- Assign to: Customer Emergency Rotation
- Title: Add the Zendesk ticket number here

No other fields need to be filled out, therefore you may then click Create Incident

Special Handling Notes

Special handling notes are documented on the customer emergencies on-call workflow. As a Support Manager, you are empowered to handle these (and other) unique situations according to your judgment. If you need help or advice, don't hesitate to escalate to unblock.

Compromised instances

We advise Support Engineers to contact a Support Manager before offering a call in the case of a compromised instance.

Support's role in these cases is to help the customer get to a good, known working state as quickly as possible. The fastest route will be to restore to a previously known good state (most often by restoring from a backup). Customers with an instance in this state will have other concerns though, and likely be in a heightened emotional state:

- How did this happen? (Which we may or may not easily be able to answer, and they should do this forensic analysis after they have restored.)
- How can we recover from this without restoring? (They can't "safely". We advise a restore to have 100% confidence in their environment.)
- What data was accessed? (This is always a challenging question, and if the compromise was led by a human, they may have covered their tracks. We may never have full knowledge. They should start restoring ASAP, and do forensics after.)

If moving towards a call is the right thing to do, consider joining the call before (or instead of) the engineer to communicate the scope of what can be accomplished.

Example framework for a call we establish (or a bridge call the customer is leading):

> Hi customer. Based on the ticket it sounds likely that your instance is compromised. In cases like these we've prepared a set of best practices ([GitLab internal link](#)) to help you get back up and running as quickly as possible. We're here to support and advise where GitLab is concerned. Unfortunately, GitLab cannot provide a one-size-fits-all solution or comprehensive checklist to completely secure a server that has been compromised. GitLab recommends following your organization's established incident response plan whenever possible.

> The first step is to shut down the instance, create a new one at the same version, and restore your most recent backup. This ensures you are operating on a "clean" environment, where you have confidence that all the software installed is unmodified. Please get that process started; we are monitoring this ticket with HIGH priority. If you have any problems getting set up or restoring, please let us know in the ticket immediately.

> After your new instance is set up, you need to upgrade to a more recent version of GitLab before you expose this server to the public Internet. If you have any trouble with the upgrade process, let us know in the ticket immediately.

> Finally, as described in the recovery guide previously sent (should have been shared in the ticket via the Compromised Instance Zendesk macro, you should do an audit of the integrity of GitLab itself: checking for any users, code, webhooks, runners or other settings that you did not enable yourselves. If you have any questions, please let us know in the ticket.

> I'm going to leave the call now, but rest assured that we're on standby and ready to help as you work through this.

SECTION: support/workflows/test_env.md

Selecting the Right Testing Environment

Support Engineers need testing environments to learn how to support GitLab and also to replicate customer issues.

This page explains the main choices available to you. Some guidelines:

1. You should choose a way to spin up a specific version of GitLab quickly so that you can replicate customer issues. The current recommended way to do this is to use GitLab Sandbox Cloud. Each specific module that you'll be using has its own README.md in the deployment projects, so make sure to give that a read as well.
1. Testing locally is also recommended - configuring Docker Engine / VM based system (details below) to your taste will let you quickly spin up a specific GitLab version.
1. You will need licenses for all self-managed tiers so you can match the features available with your customer's features - see the next section.
1. For most testing, a single box Omnibus installation will be fine.
1. If you need a more complex environment, the following tools can help:
 - GitLab Environment Toolkit
 - GitLab Support Setups via local (Virtualbox, VMWare, libvirt) VMs
1. For K8s Helm installations, we recommend using GKE.
1. If you need to replicate specific cloud provider environments (e.g. for a scaled architecture), see the sections on GCP, AWS and Azure below.
1. Consider joining the spttesting Slack Channel to share your own tips and tricks for testing environments.

Have fun!

Testing Environment License

If you require a license for testing:

- For a self-managed instance, use the Support Super Form.
- For GitLab.com, individual test namespaces with licenses pre-applied have been provisioned as part of your onboarding.

Please keep in mind you can't generate licenses for customers, only for your own testing purposes.

AWS Marketplace Resources

Note that when using some AWS Marketplace resources, free trials (including GitLab Ultimate) may automatically renew. You should always prefer using test licenses, but ensure that you cancel any AWS Marketplace subscriptions if you trial anything for testing purposes.

Testing on GitLab.com

As noted in Testing Environment License, you will have received licensed namespaces for testing on GitLab.com as part of your onboarding.

You will be added as Owner in these groups and can make changes at-will, including inviting customers to specific projects as you collaborate with them.

- Avoid making the namespace public.
 - Instead create a subgroup or project within your test namespace or a namespace unique to the customer case and assign them sufficient permissions to invite collaboration.
- Avoid granting a customer permanent access.
 - Instead set an access expiration date when you invite a customer.
- Avoid using access tokens for your main GitLab account: a leak in a test project may not be automatically detected and can be used to traverse sensitive company namespaces.
 - Instead try to use Project Access Tokens or Group Access tokens. Otherwise, create a test account and use personal access tokens for it (If you do this, make sure the token is set to expire within a maximum of 2 days).

Cloud Testing Environments

You can create (ephemeral) testing environments. We recommend using the Sandbox Cloud Realm at gitlabsandbox.cloud for doing so.

You're free to create any testing environments that you need in order to perform your role, however be advised that:

- Cloud resources are not free: please delete or shutdown any instances you aren't using.
- Multi-node reference architectures are very expensive to run, so be particularly mindful of not leaving them active longer than you need to.
- You shouldn't expose your instances to the public Internet or run without TLS. Instances vulnerable to known RCEs or other exploits detected through automated scans will be shut down without warning. Please review Securing Cloud Testing Environments before you create your first instance.

GCP

GCP resources can belong to different GCP projects. There is a number of GCP projects available to you to create your resources in.

GitLab Sandbox Cloud for GCP (preferred)

If you need flexibility for creating test environments, the GitLab Sandbox Cloud allows for creating a personally-owned GCP projects. You can create test resources using the GCP console, or gcloud command line tool. If you need to replicate any of the Reference Architectures, it's recommended that you use the GitLab Environment Toolkit.

Note: Please remember to shut down resources that you are no longer using.

We are now using automation scripts to shutdown resources over the weekend. To exclude your resources from being shutdown you'll need to add the instance-ttl-bot-ignore label to those resources.

Managing your GCP resources automatically

You can use Terraform Environments to automatically manage GCP resources in your personally-owned GCP projects. If you are after a simple Omnibus machine with a runner, select support-resources-template-v2- template. Feel free to also explore other available Project templates to deploy GitLab Environment Toolkit or create a GKE cluster that already has GitLab installed through Helm chart.

Note that these environments are ephemeral.

Check out a this demo video for details walk-through.

Other GCP Projects

Warning: You may also have access to the gitlab-internal and gitlab-support GCP projects. You should use GitLab Sandbox Cloud instead of creating resources in these projects.

Note: Please remember to shut down or delete any resources that you are no longer using.

GCP GKE Kubernetes Cluster

Please use your GitLab Sandbox Cloud GCP project to create a GCP Kubernetes (GKE) cluster. You can create a GKE cluster manually from the console or you can use the Support GKE cluster template (for an empty GKE cluster) or Support GitLab GKE cluster template (for a GKE cluster with a GitLab helm deployment) from your GitLab Sandbox account. Click here for more information on spinning up resources with terraform in the GitLab Sandbox.

Note: If you are using GKE to test GitLab Runners, note that GitLab Runners require the use of RBAC roles in GCP. The support-resources GCP project does not allow the user enough permission to create the required roles, but you can do it with your personal GitLab Sandbox Cloud GCP project

<details>

<summary>Open me for instructions on how to manually create GKE on your own project.</summary>
<div markdown="1">

1. Ensure you have selected your own project at the top of GCC.
1. Open the navigation menu at the top of GCP
1. Select Kubernetes Engine > Create Cluster from the dashboard.
1. Enter a name, select a zone, and choose the default static master version unless you have a specific reason to use an alternative version. It's important to use a server version that will match your kubectl client version.

All of the remaining options can be left as their default settings unless you have a need to add customization to your cluster. Of note, the Maximum Pods per Node option directly correlates with the CIDR assignment of your node(s).

Connecting to, and configuring, your cluster can be done locally using gcloud and kubectl. Or you can use the Google Cloud Shell. Clicking Connect in GCP will provide the command to run locally for you to copy and paste, or let you open Cloud Shell immediately. Using docker images is also an option for a quick deployment of all tools locally.

</div>

</details>

AWS Testing Environment

Please use Sandbox Cloud to create credentials for AWS.

EKS testing Environment with Helm

<details>

<summary>Open me for instructions on how to quickly create an EKS deployment using Helm</summary>

<div markdown="1">

The following guide is designed to help you quickly deploy a Kubernetes environment using EKS and Helm for testing purposes on this environment.

Pre-requisites:

- A domain name
- A GitLab Sandbox Cloud account

Install the following programs on your local computer:

- Helm v3 (3.3.1 or higher)
- Kubectl
- eksctl
- AWS CLI
- Certbot (If you are using other OS than OSX please look for your specific OS.)

For OS X, you can install all of the programs above by running:

```
shell
brew install certbot helm kubectl
```

Getting an AWS IAM Access Key

1. Log in to your Sandbox Cloud
1. Log into your AWS individual account in Sandbox Cloud
1. Create an IAM Access key

When the key has been created successfully, you'll get an Access key ID and a Secret key.

Configure locally your AWS IAM Access Key

Go to your terminal to configure the AWS CLI to use the IAM Access Key:

1. Type `aws configure`
1. Enter the Access key ID and Secret key when prompted
1. Optionally, set a region
1. Choose the output format (This controls what format the AWS CLI will use when printing out data. Options include json, yaml and text.)

EKS Setup

A bootstrap script has been created to setup an easier and automated cluster creation using CloudFormation.

Clone this project locally and navigate to `gitlab/scripts` and execute `./eksbootstrapscript up`. This script will use a CloudFormation stack to deploy all the required resources for your cluster so it will take a while to complete. After successful completion you will see something similar to this output:

NOTE: If the Kubernetes cluster version is ≥ 1.23 , you must run `./eksbootstrapscript adddevscsidriver` prior to deploying GitLab to the cluster.

```
shell
2022-07-18 16:47:40 [.] node "ip-192-168-63-211.us-east-2.compute.internal" is ready
2022-07-18 16:47:41 [.] kubectl command should work with "/Users/cats/.kube/config", try
'kubectl get nodes'
2022-07-18 16:47:41 [.] EKS cluster "gitlab-cluster" in "us-east-2" region is ready
```

Once your cluster is ready create a certificate for your domain using Certbot by running:

```
shell
sudo certbot --server https://acme-v02.api.letsencrypt.org/directory --manual --preferred-
challenges dns-01 certonly -d "<DOMAIN NAME>"
```

Certbot will create a TXT DNS record that you need to add on your DNS Registrar so that your domain will have a valid HTTPS certificate to be used by your GitLab instance.

After your record had been added to your registrar create a secret for your cluster to use the certificate by running:

shell

```
sudo kubectl create secret tls tls-star-<SECRET NAME> --cert=/etc/letsencrypt/live/<DOMAIN NAME>/fullchain.pem --key=/etc/letsencrypt/live/<DOMAIN NAME>/privkey.pem
```

(Optional) If you also want to have GitLab Pages in your cluster then you will need to create another certificate for your domain and another secret for your cluster

shell

```
sudo certbot --server https://acme-v02.api.letsencrypt.org/directory --manual --preferred-challenges dns-01 certonly -d ".pages.<DOMAIN NAME>"
```

shell

```
sudo kubectl create secret tls tls-star-pages-<SECRET NAME> --cert=/etc/letsencrypt/live/pages.<DOMAIN NAME>/fullchain.pem --key=/etc/letsencrypt/live/pages.<DOMAIN NAME>/privkey.pem
```

Helm Setup and Chart deploy

- Add the GitLab repository charts to your Helm environment

shell

```
helm repo add gitlab https://charts.gitlab.io/
```

- Update your repository to the latest version

shell

```
helm repo update
```

- Create a values.yml file with the desired values you wish to deploy your instance. (A sample file can be found [here](#))
- Deploy GitLab using

shell

```
helm install gitlab gitlab/gitlab -f <values.yml>
```

- To get the loadbalancer hostname of your GitLab deployment run

shell

```
kubectl get ingress/gitlab-webservice-default -ojsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

- Create a CNAME record with the name gitlab pointing to your loadbalancer hostname to access your instance on HTTPS.
- (Optional) If you want to setup GitLab Pages create another CNAME record using .pages pointing to the same loadbalancer hostname.
- Get your initial root password by executing

shell

```
kubectrl get secret gitlab-gitlab-initial-root-password -ojsonpath='{.data.password}' | base64 --decode ; echo
```

:tada: Your GitLab instance is up and running!

Once these steps are completed you can visit your GitLab instance using gitlab.<DOMAIN NAME>.

Destroy and recreate your environment

- Once you're done using your EKS instance run ./eksbootstrapscrip down to destroy all resources used.
- When creating a new cluster you will need to run:

1. ./eksbootstrapscrip up.
1. Create Kubernetes secrets as before.
1. Deploy your Helm chart.
1. Update your CNAME with the new loadbalancer in your registrar.

Note: You might need to wait for your TTL to reset after you updated your loadbalancer in your registrar before accessing the instance.

</div>

</details>

OpenShift Testing Environment using ROSA

One option for creating an OpenShift cluster to run the GitLab and GitLab Runner Operators is to use the Red Hat OpenShift Service on AWS (ROSA).

Installation is performed from your laptop using the rosa utility which collects cluster configuration details and then runs a scripted deployment of an OpenShift cluster in AWS. The installation typically takes 30-45 minutes.

The key steps are as follows:

Prerequisites:

1. Local AWS command line access is configured on your laptop by setting the AWSREGION, AWSACCESSKEYID and AWSSECRETACCESSKEY environment variables to access your AWS sandbox account.
1. The AWS ROSA prerequisites are met - in the AWS Console, search for the Red Hat OpenShift

Service on AWS service, click on

Get started and ensure ROSA is enabled and the Service Quotas and ELB service-linked role requirements are met.

1. A personal Red Hat account - register a new personal account at <https://www.redhat.com/wapps/ugc/register.html>.

Cluster Setup:

1. Browse to <https://console.redhat.com/openshift/create/rosa/getstarted>.

1. Log in using your Red Hat account. This will take you to the Hybrid Cloud Console.

1. Download and install the ROSA CLI tool for your local OS (step 1).

1. Login to the ROSA CLI with your Red Hat account token and create AWS account roles and policies as described on the setup page (step 2).

1. From your laptop, run `rosa create cluster` to start the installation. You will be prompted for quite a few settings - accept the offered defaults for all except the following:

- Cluster name - choose a name for your cluster
- Create cluster admin user - select Yes
- Openshift version - enter desired OpenShift version
- AWS region - enter desired AWS region

1. The installation process will start, and a message will appear prompting you to run commands to create the operator-roles and

oidc-provider in order for the installation to continue. You need to run these now or the installation will pause indefinitely.

Accept the default options when prompted.

1. Record the admin user name and password that were displayed during the setup step.

1. You can monitor the progress of the installation and the installation log from the Hybrid Cloud Console Cluster List page. Note

that multiple connection errors will be logged while services are being provisioned - these are expected.

1. While the cluster is installing you can download the OpenShift CLI tool (oc) from the link provided on the cluster status page, if required.

Logging in to the OpenShift cluster

Once the cluster state is ready you can log into the cluster console from the Hybrid Cloud Console by selecting the cluster from Cluster List and clicking on the Open Console button.

However you will need to wait an additional 5-10 minutes for the setup of the default identify provider and TLS certificates to complete and the cluster-admin identify provider button to appear first on the login page. If you are only presented with username and password fields to fill in wait a few more minutes and try browsing to the console login page again.

Once you see the cluster-admin button click on it and you will be prompted for the admin user

and password you recorded earlier. You are now logged into the cluster.

Accessing the OpenShift cluster from the command line

To access the cluster from the command line you first need to log into it using the oc command. From the cluster console, click on the cluster-admin drop down at the top right and then on Copy login command. You will be prompted to log in as the cluster admin again, and then shown a Display token link. Click on the link and copy the oc login command that is displayed.

Run the oc login command on your laptop with the supplied token - you should then be able to run other commands such as oc get pods -A.

Destroying the cluster

ROSA clusters aren't cheap to run and so to reduce AWS costs the OpenShift cluster should be destroyed as soon as it is no longer needed (even if that means building a new one in a couple of days) - this is done from the command line by running `rosa delete cluster --cluster=mycluster`.

Azure Testing Environment

For Group SAML/SCIM (GitLab.com) testing, shared account credentials can be located within the 1password entry Azure Active Directory Sandbox (SAML Testing). This level of access should be sufficient for the majority of test cases.

If you need access to the shared infrastructure team's Azure account for testing AKS and similar features, please create an access request and tag your manager.

For features not available in either instance, please create your own trial for Azure.

Okta Testing Environment

If you need an Okta sandbox environment to test SAML and SCIM, please go to <https://developer.okta.com/signup/> and enter your credentials for a free developer instance where you can perform all your tests. This is a full featured environment so you should be able to add and remove applications, and perform all tests as if you were in a regular production instance.

Google Workspace Testing Environment

The IT team has a Google Workspace test environment which the Support team can use. The credentials for 1 admin and 1 user account can be found in 1password as:

- Google Workspace Admin Test Account
- Google Workspace User Test Account

If you need to create more user test accounts, please delete them afterwards. There is a seat limit in the test environment.

When creating apps, please include your name or the ticket number in the app name. Note that all apps should be considered temporary and may be deleted 2 weeks after creation.

LDAP Testing Environments

For testing LDAP integrations with a self-managed GitLab instance, you may consider any of these options:

1. GDK.
1. Docker container.
1. Jumpcloud (Free for up to 10 users).

DNS for test instances

See the handbook for IT policies about Domain Names and DNS Records.

If you wish to test resources using a real domain name (instead of an IP address, e.g. for testing TLS certificates), you can purchase a domain name of your choice (without GitLab trademarks) that is paid for by GitLab through your Sandbox Cloud AWS account or GCP project. Instructions and guidelines can be found [here](#).

Securing Cloud Testing Environments

Test instances are, by default, publicly accessible on the Internet. Often, we need to test specific versions or configurations that may be vulnerable to remote compromise. It is your responsibility to secure your test instances to prevent them from being compromised and used to further attack our cloud environment.

The GitLab Red Team regularly scans GitLab's cloud environments for publicly accessible instances with known vulnerabilities. Instances vulnerable to known RCEs or other exploits detected through automated scans will be shut down without warning.

IP Filtering

A highly effective way to secure your cloud instances is to apply the concept of IP filtering for each test instance you create whether its a GitLab instance or otherwise. For the majority of cases, this means source IP filtering from one or more CIDR block ranges ensuring that only certain IPs and integrations can interact with the GitLab instance, therefore reducing the attack surface of the GitLab organization as a whole.

If you don't know your current IP address to use for source IP filtering, you can utilize services like whatismyipaddress.com or ipinfo.io to retrieve it. Though configuring IP filtering is currently a manual process for each cloud service, discussions are underway at the time of writing regarding how best to automate this practice to reduce complexity and manual processes needed in some cases. The steps to implement IP filtering will differ per cloud environment. Below you can find a detailed guide maintained by the support engineering team.

- Support Engineering Step-by-Step Guide to Implementing IP Filtering

In addition, you can find official, platform-specific documentation of features involved in implementing IP filtering:

- Google Cloud
- Amazon Web Services (AWS)
- Azure

TLS

To be in line with GitLab's encryption policy, TLS should also be implemented on public-facing testing resources.

For GitLab instances specifically, it's recommended that LetsEncrypt is manually enabled for Omnibus installs. First, you'll need a domain assigned to your external IP regardless of your cloud platform. Enabling LetsEncrypt for Omnibus installs is well documented at the link above.

Self-Signed Certificates

Implementing TLS on any test instance that includes a login page can be done with a self-signed certificate if desired. Self-signed certificates are free, suitable for testing environments, and encrypt ingress and egress traffic with the same ciphers as paid certificates. The down-side is that self-signed certificates are not trusted by any browser or operating system and will therefore warn users of the risks when accessing a site that utilizes a self-signed (untrusted) certificate. If external parties will be accessing your instance that should rely on your TLS implementation, it's best to include a signed certificate from a legitimate certificate authority.

Self-signed certificates can be generated with a tool like mkcert. Once mkcert has been installed, you can this command to generate a certificate file and a key file for gitlab.example.com:

```
sh
mkcert gitlab.example.com
```

The command's output will tell you the filenames that are generated:

```
>sh
> The certificate is at "./gitlab.example.com.pem" and the key at "./gitlab.example.com-
key.pem" .
>
```

For instructions on using self-signed certificates on your test instances, please review the GitLab docs on Configure HTTPS manually and the documentation available from these cloud service providers:

- Google Cloud

- AWS
- Azure

Patching against known vulnerabilities

Running a Support test instance with a known vulnerability can be a security issue. When running a test instance, you should patch against known vulnerabilities if possible.

For example, GitLab versions from 11.9 to 13.10.2 are vulnerable to CVE-2021-22205.

If you're running GitLab version 12.6 to 13.10.2, patch against this vulnerability by running the following commands for each instance:

```
sh
sudo su
cd ~
curl -JLO https://gitlab.com/gitlab-org/build/CNG/-/raw/master/gitlab-ruby/patches/allow-only-tiff-jpeg-exif-strip.patch
cd /opt/gitlab/embedded/lib/exiftool-perl
patch -p2 < ~/allow-only-tiff-jpeg-exif-strip.patch
```

For GitLab versions 11.9.0 to 12.5.x, you can "patch" the vulnerability by replacing the exiftool binary with the following commands:

```
sh
sudo rm -f /opt/gitlab/embedded/bin/exiftool
sudo printf '!\bin/bash \n\ncat -' > /opt/gitlab/embedded/bin/exiftool
sudo chmod a+x /opt/gitlab/embedded/bin/exiftool
```

Local Testing Environments

Docker

If you'd like to use Docker Desktop for Mac a subscription is required for business use. Please review the Docker Desktop handbook page to find more information on how to obtain a license as well as a list of recommended alternatives.

In the mean time, consider using a Cloud or local VM with Linux Engine for testing Docker environments.

Note that on Macs with M1 / Apple Silicon, running GitLab in Docker is not working properly for now. Check out UTM below as an alternative for a local setup on your M1 Mac.

Docker Machine

Since Docker Toolbox has been deprecated, Docker Machine has to be downloaded and installed manually. Use the following commands to install or upgrade Docker Machine separately:

sh

```
$ curl -L https://github.com/docker/machine/releases/download/v0.16.2/docker-machine-  
-s-uname -m >/usr/local/bin/docker-machine && \  
  chmod +x /usr/local/bin/docker-machine
```

VMWare

This guide involves configuring and setting up VMWare and Docker locally and assumes you're using macOS.

Install VMWare (paid)

1. Navigate to VMWare store, and then purchase VMWare Fusion 10 (for macOS) (or current version).
1. Download VMWare Fusion 10 (for macOS) using the provided link.
1. Install VMWare Fusion 10.
1. Launch VMWare Fusion.
1. When prompted, enter the license details.

UTM (Apple M1 Compatible)

UTM is an open source QEMU-based virtualisation software for MacOS. It currently provides support for Apple silicon hardware, so may be useful for engineers on Apple M1 devices to deploy local virtual machines.

Install UTM

1. Navigate to the UTM home page and select 'Download'.
1. Once installed, follow the instructions for an ARM-based installation of Ubuntu
1. Start the VM within the UTM interface.

VirtualBox (Incompatible with Apple M1)

This guide involves configuring and setting up VirtualBox and Docker locally and assumes you're using macOS. VirtualBox does not currently support Apple M1 CPUs.

Install VirtualBox

Oracle VM VirtualBox is a free and open-source hosted hypervisor for x86 virtualization.

1. Navigate to VirtualBox.
1. Download the latest version of the software for your operating system.
1. Install VirtualBox.

Note The following list of commands can be saved as bash script for quickly spinning up new instances

Vagrant

Install Vagrant

From Introduction to Vagrant

> Vagrant is a tool for building and managing virtual machine environments in a single workflow

Vagrant encapsulates the local VM apps VMWare and Virtual along with libvirt.

To install Vagrant, go to [tutorials/vagrant/getting-started-install](#)

Once installed, support/toolbox has two projects which you can explore for local GitLab and tools setup.

- GitLab Support Toolkit

> Support toolkit to help manage GitLab inventory and additional services via docker containers.

- GitLab Support Setups

> Provide a common provisioning and directory structure for various support setups including GitLab with connected GitLab Runners.

Multipass

Install multipass

Multipass is a tool to generate cloud-style Ubuntu VMs quickly on Linux, macOS, and Windows. This method is similar to Vagrant.

It can be installed using brew or the package installer.

NOTE: Some Mac users may experience a long standing bug where the MacOS firewall prevents Multipass from functioning consistently. Use macOS 13.3.1 or above to avoid this issue.

Once installed, use multipass help to get an idea of what it can do. The general format is `multipass <command> <name>`.

For a solution that uses multipass see [GitLab-in-a-VM](#). It automates a lot of stuff regarding setting up the VMs and making sure the instance is trusted between the GitLab runner and the GitLab instance itself.

Examples

Omnibus

1. Make sure you download and install multipass.

1. Create a script called `install-omnibus.sh` inside of `~/mpmount` (or your preferred mount directory) with the following content:

```

bash
!/bin/bash

echo "Starting script..."
sudo apt-get update
sudo apt-get install -y curl openssh-server ca-certificates
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh | sudo
bash
export GITLABROOTPASSWORD="yourrootpassword"
export EXTERNALURL="$(hostname -I | awk '{print $1}')"
echo $EXTERNALURL
echo "Please enter the GitLab version (ex. 15.1.1):"
read GLVERSION
apt install -y "gitlab-ee=$GLVERSION-ee.0"

```

Please note: You need to replace the above password entry "yourrootpassword" with an alphanumeric password which is not a series of words and phrases otherwise you will encounter the error message "Password must not contain commonly used combinations of words and letters".

1. Run the following one-liner. If you'd like to increase/decrease memory or disk, replace with the appropriate values. You can use `multipass help launch` for more details on this command.

```

bash
multipass launch --cpus 4 --memory 8G --disk 10G --name gitlab-omnibus && multipass mount
~/mpmount/ gitlab-omnibus:/mpmount && multipass exec gitlab-omnibus -- sh -c 'sudo sh
/mpmount/install-omnibus.sh'

```

1. You're finished! Use `multipass shell gitlab-omnibus` to access your instance. Additionally:

- `multipass list` or `multipass ls` to see all instances
- `multipass stop gitlab-omnibus` to stop the instance
- `multipass delete gitlab-omnibus` to delete the instance
- Change the default open shell menu item by using `duti`

GitLab Runner

1. Create an executable script called `install-runner.sh` inside of `~/mpmount` (or your preferred mount directory) with the following content:

```

bash
!/bin/bash

echo "Starting script..."
sudo apt-get update
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]

```

```
https://download.docker.com/linux/ubuntu \
$(lsbrelease -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-
runner/script.deb.sh" | sudo bash
apt-get install -y gitlab-runner
```

1. Create a new instance:

```
bash
multipass launch --cpus 2 --memory 4G --disk 5G --name gitlab-runner && multipass mount
~/mpmount/ gitlab-runner:/mpmount && multipass exec gitlab-runner -- sh -c 'sudo sh
/mpmount/install-runner.sh'
```

1. Use multipass shell gitlab-runner to access your instance.

Elasticsearch Instance

1. Create a script called install-es.sh inside of ~/mpmount (or your preferred mount directory) with the following content:

```
bash
!/bin/bash

wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o
/usr/share/keyrings/elasticsearch-keyring.gpg
sudo apt-get install apt-transport-https
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]
https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee
/etc/apt/sources.list.d/elastic-7.x.list
sudo apt-get update && sudo apt-get install elasticsearch
sudo /bin/systemctl daemon-reload
sudo /bin/systemctl enable elasticsearch.service
sudo systemctl start elasticsearch.service
cat << EOF > /etc/elasticsearch/elasticsearch.yml
path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch
network.host: $(hostname -I | awk '{print $1}')
http.port: 9200
discovery.seedhosts: ["$(hostname -I | awk '{print $1}'), "elasticsearch"]
EOF
sudo systemctl restart elasticsearch.service
```

1. Create a new instance:

```
bash
```

```
multipass launch --cpus 2 --memory 8G --disk 5G --name gitlab-es && multipass mount  
~/mpmount/ gitlab-es:/mpmount && multipass exec gitlab-es -- sh -c 'sudo sh /mpmount/install-  
es.sh'
```

1. Use multipass shell gitlab-es to access your instance. You can use multipass ls to get the IP address and connect to it in your browser at port :9200 to verify it is working. Alternatively, run curl ipaddress:9200 from your GitLab Omnibus instance to confirm you can reach your Elasticsearch instance.

Creating GitLab test instance

Creating settings variables

```
sh  
export SSHPORT=2222  
export HTTPPORT=8888  
export VERSION=11.9.9-ee.0  
export ENVNAME=gitlab-test-env  
export CONTAINERNAME=gitlab-test-11.9
```

Create new docker host

This command will create a new VirtualBox virtual machine called gitlab-test-env that will act as a docker host.

- CPUs: Same as host (-1)
- RAM: 4GB
- Name: gitlab-test-env
- Driver: virtualbox

```
sh  
docker-machine create \  
--virtualbox-cpu-count -1 \  
--virtualbox-memory 4096 \  
--virtualbox-disk-size 30000 \  
--driver virtualbox $ENVNAME
```

- Resource:
<https://web.archive.org/web/20210619101324/https://docs.docker.com/machine/drivers/virtualbox/>

Creating GitLab test instance with docker machine

Connect your shell to the new machine

In this example we'll create a GitLab EE 11.9.9 instance.

First connect the docker client to the docker host you created previously.

```
sh
eval "$(docker-machine env gitlab-test-env)"
```

You can add this to your ~/.bashprofile file to ensure the docker client uses the gitlab-test-env docker host. You can use echo to do so:

```
sh
echo 'eval "$(docker-machine env gitlab-test-env)"' >> ~/.bashprofile
```

Get the available tags for GitLab

Optionally replace the ee in the URL with ce

```
sh
wget -q https://registry.hub.docker.com/v1/repositories/gitlab/gitlab-ee/tags -O - | sed -e
's/[]//g' -e 's/"/"/g' -e 's/ / /g' | tr '}' '\n' | awk -F: '{print $3}'
```

Create new GitLab container

- HTTP port: 8888
- SSH port: 2222
 - Set gitlabshellsshport using --env GITLABOMNIBUSCONFIG
- Hostname: IP of docker host
- Container name: gitlab-test-11.9
- GitLab version: EE 11.9.9-ee.0

<!-- Set up container settings

```
sh
export SSHPORT=2222
export HTTPPORT=8888
export VERSION=11.9.9-ee.0
export NAME=gitlab-test-11.9
-->
```

Create container

```
sh
export IP=$(docker-machine ip $ENVNAME)

docker run --detach \
--env GITLABOMNIBUSCONFIG="externalurl 'https://$IP:$HTTPPORT';
gitlabrails['gitlabshellsshport'] = $SSHPORT;" \
--hostname $IP \
-p $HTTPPORT:$HTTPPORT -p $SSHPORT:22 \
--name $CONTAINERNAME \
gitlab/gitlab-ee:$VERSION
```

Connect to the GitLab container

Retrieve the docker host IP

```
sh
echo $IP
example output: 192.168.151.134
```

- Browse to: <<https://192.168.151.134:8888/>>

Note: The container might take a few seconds to spin up and become accessible via the browser.

Execute interactive shell/edit configuration

```
sh
docker exec -it $CONTAINERNAME /bin/bash
```

```
sh
example commands
root@192:/ vi /etc/gitlab/gitlab.rb
root@192:/ gitlab-ctl reconfigure
```

How to update gitlab.rb values with sed

For example, to set the gitlabshellssh port on a container named gitlab-ee to port 2222

```
sh
docker exec -it gitlab-ee \
sed -i "s/.gitlabshellsshport/.gitlabrails['gitlabshellsshport'] = 2222/g"
/etc/gitlab/gitlab.rb
```

```
docker exec -it gitlab-ee gitlab-ctl reconfigure
```

Resources

- <<https://docs.gitlab.com/install/docker/>>
- <<https://web.archive.org/web/20210619101324/https://docs.docker.com/machine/get-started/>>
- <<https://web.archive.org/web/20210619101324/https://docs.docker.com/machine/reference/ip/>>

Windows

It may come to pass that you require a Windows environment to test a Windows Runner or even the Kubernetes Executor in a Mixed Cluster.

The options are the same as above:

- Cloud environments: GCP and AWS have Windows Server images you can spin up to connect to via RDP.
- A local environment: Microsoft provides pre-packaged Windows VMs for your hypervisor of choice.

SECTION: support/workflows/ticket-transfers.md

Introduction

This page helps guide Support Engineers (SEs) when they need to transfer a ticket to a different assignee.

Ticket transfers

GitLab Support has two different types of ticket transfers: rehomes and handovers. The following sections describe these, and the supporting concepts of satellite tickets and hot rehomes.

1. Handover ·: When a ticket is transferred from one engineer to another due to factors such as paid time off, expertise, or workload management. Handovers are usually in-region, but can also happen cross-region.

2. Rehome ·: A transfer of a ticket from the region of the SE who delivered the first response, to the customer's specified preferred region. At the end of the assigned SE's shift, they will forward the ticket to the appropriate (home) region for completion. These are quick, easy, and should be low effort.

- Satellite ·: A rehomed ticket that remains unassigned during the upcoming shift in the designated "home region" and subsequently circulates globally at least once. These are unintended and we should be working to minimize satellites.

- Hot rehome ·: A ticket currently being addressed in a different region than the customer's preferred one. The ticket then requires a rehome because the customer's temperature is increasing, which raises the urgency. The sending region makes commitments to the customer with the best intentions, but there is no time for alignment and agreement for the receiving region.

Examples of these tickets include:

- Tickets for which a call or special follow-up has been promised within the next few hours without confirming availability with the receiving region present a challenge for the Support Engineer to address without adequate preparation. (Direct to Call - DTC)
- Tickets not in their region that will trigger the Hot Rehome process due to unmet customer expectations with a STAR request (Falling Star).
- Not rehomed tickets that require additional attention. (The ticket was held outside of the preferred region for longer than 1 shift, which is raising the risk)

Hot rehome prevention

Before placing any ticket in a region other than the customer's preferred one, please ensure the following steps are completed:

Agreement: The customer must acknowledge that their initial request for a specific region will not be fulfilled and must provide their consent. Additionally, they should understand the added value associated with keeping the ticket in the current region.

Understanding: The customer must be made aware that there may be delays in responses. If it becomes necessary to transfer the ticket to the originally requested region, a transition period will be implemented to ensure the receiving engineer is adequately prepared.

As you work on the ticket, refrain from making promises on behalf of others. Instead, leave a detailed note in the ticket that outlines the customer's situation and emphasizes the intention to achieve a favorable outcome for them. If necessary, contact a manager in the receiving region for assistance. Furthermore, be considerate of the time of the receiving support engineer.

Ticket rehome (transfer to the preferred region)

GitLab Support uses a single view for all unassigned tickets, regardless of region. The tickets in the view are sorted by Ticket Weight to highlight the highest-priority issues first. Support engineers are expected to work from the top of the view down. Given this process, support engineers will frequently be taking tickets on which the customer has specified a Preferred region different from their own.

Since we aim in GitLab Support to align tickets to an accountable region, we use the following simple process to handle tickets from different regions:

First response

When providing the first response to a ticket from a different region, follow these guidelines:

1. Take assignment of the ticket so that you're notified of any replies until you initiate a rehome
1. Introduce yourself and acknowledge the customer's preferred region (Support::Out of Region::Cross-regionPreferred region clarify assignment macro is a good starting point)
1. Explain that you're providing initial assistance to ensure a timely response
1. Proceed with all appropriate initial steps to begin addressing the customer's issue
1. Inform the customer that before the end of your workday you will transfer their ticket to their preferred region for continued support unless they request that you keep it
1. If it's the end of your work day and the customer hasn't confirmed the region, proceed to the next section

Initiating a ticket rehome

Before initiating a ticket rehome, double check that you are the assignee on the ticket. If you

are not, take assignment
and submit the update before proceeding.

To initiate a ticket rehome, use the Support::Rehome::Initiate Rehome macro. This macro:

1. Removes ticket assignment
1. Applies the rehomeinitiated tag
1. Applies an internal comment indicating the rehome regions

If the ticket is waiting on a customer reply, set the ticket status to Pending. If the ticket is waiting on a reply from a support engineer, set the ticket status to Open. Submit the ticket to initiate the rehome.

Receiving a ticket rehome

When you take an unassigned ticket from the global view and you see an internal comment that states `Rehome initiated from [assignee's region] to [target region]`, you should follow these steps to receive that rehome:

1. Verify Regional Alignment: If the target region specified is your region, continue. Otherwise, leave the ticket unassigned in the view so that an SE from the target region can take it.
1. Use the Support::Rehome::Complete Rehome macro, which:
 1. Assigns the ticket to you
 1. Applies the rehomereceived tag
 1. Creates a public response with some initial text for your use
1. Edit the initial text to use your name where the placeholder is
1. Proceed with all appropriate initial steps to begin addressing the customer's issue, and replace the Next Steps Here placeholder with your full update to the customer

Keeping an out-of-region ticket

If a customer responds to your initi